

The Reproducibility Crisis of LLM Benchmarks in Software Engineering: An Industrial Perspective

Gustavo Oliva
Huawei Canada
Kingston, Canada

Abstract

The integration of Large Language Models (LLMs) into Software Engineering (SE) has spurred the creation of benchmarks like SWE-Bench and LCB to measure their capabilities. However, current practices for reporting results are failing us, leading to a reproducibility crisis. The execution of these benchmarks depends on a vast number of parameters—from the LLM’s training cutoff and inference settings to the specific versions of evaluation scaffolds—many of which go unreported. While some parameters have a greater impact than others, any one of them can alter the outcome, making it impossible for industrial teams to establish reliable baselines for their own models. This paper argues that the SE community must move beyond isolated, single-point benchmark publications. Drawing inspiration from mature practices in hardware benchmarking, we propose a paradigm shift towards a crowdsourced and continuously updated evaluation model. In this model, benchmark results are submitted by various parties, complete with detailed configuration reports, to establish a public, transparent distribution of performance data (e.g., median, average). This approach would not only expose the impact of different configurations but also create the robust, trustworthy baselines that industry desperately needs. We contend that such a shift is essential for fostering genuine progress and enabling fair comparison in the evaluation of LLMs for software engineering.

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Keyword 1, keyword 2

ACM Reference Format:

Gustavo Oliva. 2018. The Reproducibility Crisis of LLM Benchmarks in Software Engineering: An Industrial Perspective. In *Proceedings of 48th International Conference on Software Engineering (ICSE 2026)*. ACM, New York, NY, USA, 3 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ICSE 2026, Rio de Janeiro, Brazil

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

The Reproducibility Crisis of LLM Benchmarks in Software Engineering: An Industrial Perspective

1 Introduction

The rapid advancement of Large Language Models (LLMs) has created a paradigm shift in software engineering (SE), with models now capable of generating code, fixing bugs, and even designing entire software components [2]. To measure their progress, the community has developed benchmarks like SWE-Bench [3], Live-CodeBench [5], and the Aider Polyglot benchmark [1]. However, the current practices for reporting results are failing, leading to a crisis in reproducibility, transparency, and trustworthiness [?].

From an industrial perspective, this is a critical obstacle. As a company training our own LLMs, we rely on public benchmarks to establish a performance baseline. Furthermore, executing these benchmarks is a time-consuming and computationally expensive endeavor. Integrating multiple benchmarks, each with its own unique setup and non-standard parameters, is akin to a complex legacy system integration project. This lack of standardization leads to wasted resources and makes it impossible to reliably compare results. For instance, a public, crowdsourced baseline would allow us to see how far a new model’s official scores deviate from the median, providing a much-needed reality check.

In this paper, we distill our team’s practical experience into a set of five concrete recommendations for the community. We begin with a motivating example, using our own data to show how minor parameter variations cause significant swings in benchmark scores. We then present our core recommendations: (1) Mandate cryptographic hashes of model weights for verifiable integrity. (2) Require a comprehensive, machine-readable configuration file for all runs. (3) Enforce the reporting of a "default settings" baseline alongside any optimized results. (4) Champion a shift to a crowdsourced, statistical benchmarking model. (5) Issue a call for standardized benchmark engineering, treating benchmarks as robust, containerized tools with standard interfaces and resilient execution. Our goal is to move the community towards a more robust, reliable, and scientific foundation for evaluating LLMs in software engineering.

2 Motivating Example

In this section, we will present data from three separate executions of the SWE-Bench benchmark on the same Large Language Model. The first execution will use the default parameters provided by the official repository. The second will use a slightly modified inference configuration (e.g., a different temperature setting). The third will use a different version of the evaluation harness. We will show that these seemingly minor variations lead to a significant and misleading variance in the final "pass@1" score, highlighting the urgent need for the recommendations that follow.

3 Recommendations

In this section, we present five concrete recommendations derived from our industrial experience. These recommendations are designed to be practical and actionable, addressing the key challenges of reproducibility, transparency, and trustworthiness in LLM benchmarking for software engineering.

3.1 Comprehensive and Standardized Configuration Reporting

A fundamental barrier to reproducibility is the ad-hoc and incomplete manner in which benchmark configurations are currently reported. To address this, we propose a standardized, multi-part reporting structure that covers all layers of the evaluation stack. This structure should be captured in a `benchmark_spec.yaml` file to ensure clarity and consistency.

3.1.1 Model Specification. Trust in benchmark results begins with knowing precisely which model was evaluated. Currently, submissions often only specify a model's name (e.g., "Llama-3-70B"). This is insufficient, as fine-tuned variants can have vastly different capabilities. We propose that all submissions must specify the exact base model and include a cryptographic hash (e.g., SHA-256) of the model weights for any open-source model. This provides an unambiguous, verifiable fingerprint of the exact artifact used.

3.1.2 Inference Engine Parameters. The software used to serve the model can significantly impact performance. Parameters related to the deployment and optimization of the inference engine, such as the configuration of vLLM or TensorRT-LLM, are often overlooked. This includes settings like the level of quantization, data-parallel size, and other engine-specific tunings. These parameters must be explicitly reported to understand the full context of the execution environment.

3.1.3 Inference Parameters. The parameters used at the time of generation directly control the model's output behavior. While some benchmarks report basic settings like temperature, a complete picture is necessary. We recommend the mandatory reporting of all inference parameters, including but not limited to temperature, top-p, top-k, repetition penalty, and maximum token count.

3.1.4 Scaffold Parameters. The "scaffold" or "harness" that orchestrates the interaction between the model and the benchmark tasks is a major source of variability. Frameworks like OpenHands for SWE-Bench have their own configuration parameters that can dramatically influence outcomes. These settings, which control aspects like the agent's strategy or the number of interaction turns, must be treated as first-class citizens in the configuration report.

3.1.5 Execution Environment. When a benchmark is not executed within a provided container, the host environment becomes a significant source of variability. We recommend that all non-containerized benchmark runs must report a complete snapshot of the execution environment. This includes the operating system, Python version, and the specific version of critical hardware drivers like CUDA. Furthermore, a full list of pinned library dependencies, such as the output of `pip freeze`, must be included to prevent discrepancies arising from package updates.

3.2 Shift to Crowdsourced, Statistical Benchmarking

The current model of relying on a few single-point results published in papers or by model vendors is fragile and prone to cherry-picking. We advocate for a paradigm shift towards a crowdsourced, statistical model, inspired by mature fields like hardware benchmarking [4]. We envision a central, public repository where the community can submit benchmark results, each accompanied by its detailed configuration file (as per Recommendation 2). This would create a living, transparent distribution of performance data for any given model, allowing for the calculation of robust statistical measures like median and average performance. Such a system would serve as a powerful "reality check" against official claims and provide a much more reliable signal of a model's true capabilities.

Model providers, in their quest to showcase the best possible performance, often publish benchmark scores obtained from highly-tuned configurations. While these "hero run" results are valuable for demonstrating a model's peak potential, they can be misleading if presented in isolation. We recommend that any benchmark result published by a model provider must be accompanied by a result from a run using the benchmark's official, out-of-the-box default scaffold parameters. This mandatory baseline provides a stable, common reference point, allowing the community to fairly assess the true "alpha" generated by custom tuning versus the model's core capabilities under a standardized setting.

3.3 Standardized Benchmark Engineering

From an industrial perspective, integrating and running multiple benchmarks is a significant software engineering challenge. Each benchmark is often a standalone, idiosyncratic system, making scaled evaluation a complex and costly integration project. We issue a call for the community to treat benchmarks as robust, interoperable software tools. This includes standardizing their interfaces, such as supporting a common set of inference parameters. It also means engineering for resilience and efficiency, for example, by immediately serializing the result of each task to prevent catastrophic data loss from a single failure. Finally, benchmarks should be distributed as containerized applications (e.g., using Docker) to ensure they can be run in a safe, isolated, and easily manageable environment.

4 Related Work

5 Conclusion

References

- [1] Paul Gauthier. 2025. Aider LLM Leaderboards. <https://aider.chat/docs/leaderboards/>. Accessed: 2025-09-19.
- [2] Yuxia Hou, Yanjie Li, Yicong Zhang, He Wang, and Yao Yu. 2023. Large language models for software engineering: A systematic literature review. *arXiv preprint arXiv:2308.10620* (2023).
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Solve Real-World Software Engineering Problems?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum/?id=vfr2p51s4i>
- [4] Standard Performance Evaluation Corporation. [n. d.]. Standard Performance Evaluation Corporation (SPEC). <https://www.spec.org/>. Accessed: 2025-09-19.
- [5] Zhenghao Zhang, Zhaowei Zhang, Le-Cong Dinh, Zihan Wang, Yijun Yang, Chen-Yu Hsu, Yao-Yuan Yang, Yung-Hsiang Lu, Chen-Hao Hsu, An-Chieh Cheng, Cheng-Kuang Lee, and Yu-Chung Hsiao. 2024. LiveCodeBench: A Toolkit for Building and Evaluating Live-Coding Benchmarks on Real-World Tools and APIs. *arXiv:2407.09695* [cs.SE]

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009

233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290

291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348